

Architectural Transformation Pipeline: Engineering a Real-Time 3D-to-2D Floor Plan Engine and Interactive Web CAD Platform

Transforming heterogeneous three-dimensional spatial assets—ranging from unorganized dense point clouds and polygon meshes to Building Information Modeling (BIM) containers—into dimensionally accurate, topologically sound two-dimensional floor plans presents an intricate intersection of computational geometry, machine perception, and software systems engineering. While consumer capture platforms demonstrate near-instantaneous floor plan generation, reproducing this behavior across arbitrary 3D model formats within an open-source, commercially viable, and web-distributable environment requires establishing clear architectural boundaries between spatial parsing, cross-sectional extraction, geometric regularization, and parametric canvas interaction.

1. Deconstructing Real-Time Capture Systems: The Polycam Architecture

The perceived speed of mobile spatial capture applications such as Polycam, Canvas, or Magicplan stems from an architectural paradigm fundamentally distinct from post-hoc mesh slicing. Understanding this distinction clarifies why arbitrary 3D model ingestion behaves differently from real-time spatial scanning and establishes the engineering benchmarks necessary to maximize web processing throughput.

Mobile scanning engines do not capture an arbitrary high-density polygon mesh and subsequently solve complex topological cross-sections. Instead, they leverage hardware-accelerated spatial APIs—predominantly Apple’s RoomPlan framework, which is coupled directly to onboard time-of-flight LiDAR sensors and ARKit visual-inertial odometry¹. The scanning runtime tracks 60 Hz device motion vectors concurrently with depth sensor readings, executing on-device neural network inference directly on the Apple Neural Engine to identify architectural surfaces in real time⁴.

Architectural Attribute	Real-Time Mobile SLAM (Polycam / RoomPlan)	Static Multi-Format 3D Ingestion Engine
Input Modality	Synchronized 60 Hz RGB video, IMU pose estimates, and sparse LiDAR point bursts ⁴ .	Static triangle meshes (OBJ, GLTF, FBX, IFC) or dense point clouds (PLY, LAS, PCD, E57) ⁶ .

Compute Location	Local edge hardware (Neural Engine / Mobile GPU) executing concurrent with capture ⁴ .	Client-side browser thread (WebAssembly/WebGPU) or remote server compute worker (C++, Python, .NET).
Semantic Ingestion	Instantaneous classification of architectural primitives (walls, doors, windows, furniture) ⁴ .	Post-hoc geometric extraction, planar segmentation, and topological cycle reconstruction ⁶ .
Latency Profile	Sub-second; parametric bounding boxes (CapturedRoom) are updated incrementally ⁴ .	Multi-stage pipeline: parsing ($O(N)$), spatial indexing ($O(N \log N)$), slicing ($O(K \log N)$), and vectorization ⁷ .
Output Representation	Parametric structural vectors with predefined orthogonal relationships ⁴ .	Raw cross-sectional segment collections requiring regularization and gap closure ⁶ .

When Apple RoomPlan finishes a capture session, it outputs a structured CapturedRoom data object, serializable into a standardized JSON payload that contains explicit wall boundaries, metric thickness values, transformation matrices, and relative offsets for embedded openings such as doors and windows⁴. Because parametric inference occurs progressively throughout sensor translation, the conversion step is negligible.

In contrast, an arbitrary model uploaded to a web platform lacks sensor priors, calibrated camera trajectories, and semantic object metadata. The incoming asset may be an unoriented photogrammetric mesh containing millions of degenerate triangles, a hollow laser scan, or an unindexed BIM container⁶. Consequently, matching the responsiveness of mobile capture tools requires designing an optimized computational geometry pipeline that performs spatial indexing, plane-mesh intersection, and polygon reconstruction with minimal CPU overhead.

2. Permissive Open-Source Foundation and Commercial Viability

Commercial deployment requires a technology stack free from copyleft licensing encumbrances. Utilizing libraries governed by the GNU General Public License (GPL) or the Affero General Public License (AGPL) exposes proprietary platforms to intellectual property risks. Conversely, libraries licensed under the MIT, Apache-2.0, BSD, or Boost Software License

(BSL-1.0) allow unrestricted commercial redistribution, closed-source derivative works, and seamless SaaS integration¹⁰.

Subsystem	Library Candidate	Runtime / Language	License	Primary Architectural Function
3D Mesh I/O	Open3D / Trimesh / Assimp	C++ / Python / WebAssembly	MIT / BSD-3-Clause	Parsing OBJ, PLY, GLTF/GLB, STL, and DXF assets ¹³ .
BIM Processing	IfcOpenShell (IfcConvert)	C++ / Python	LGPL-3.0 (Isolated Service CLI)	Direct extraction of architectural primitives (IfcWall, IfcDoor) to SVG ⁹ .
Solid Slicing Kernel	Manifold (manifold3d / manifold-3d)	C++ / Wasm / Python	Apache-2.0	Guaranteed-m manifold geometric booleans and planar cross-section extraction ⁷ .
2D Polygon Topology	Clipper2 (pyclipr / clipper2)	C++ / C# / Python / Wasm	Boost (BSL-1.0)	Planar polygon clipping, offsetting, line-buffering, and winding resolution ¹⁰ .
Graph Modeling	Custom Half-Edge / NetworkX	TypeScript / C# / Python	MIT / BSD	Managing wall junctions, collinear edge merges, and room cycle extraction ¹⁷ .

Interactive Canvas	Konva.js (react-konva)	TypeScript	MIT	Retained-mode 2D drafting canvas, node-dragging, and snapping visualizers ¹⁹ .
3D Web Rendering	Three.js	TypeScript / JavaScript	MIT	Hardware-accelerated WebGL/WebGPU extrusion and spatial visualization ²⁰ .
Agent Orchestration	Cursor SDK (@cursor/sdk)	TypeScript / Python	Commercial API	Autonomous schema validation, natural language mutations, and automated repairs ²² .

The foundation of the computational geometry layer rests on Emmett Lalish's Manifold library⁷. Traditional Constructive Solid Geometry (CSG) algorithms suffer from floating-point roundoff errors, frequently producing non-manifold geometry, self-intersecting loops, or collapsed faces when performing boolean operations and planar slicing²⁵. Manifold circumvents these numerical vulnerabilities by divorcing exact topological connectivity from inexact geometric vertex coordinates, enforcing guaranteed-manifold topological outputs across all intersections⁷. Available as a native C++ library, a Python package (manifold3d), and a WebAssembly package (manifold-3d), it provides specialized slicing routines that intersect 3D solids with arbitrary planar boundaries to generate valid planar cross-sections⁷. For 2D polygonal operations, Angus Johnson's Clipper2 library serves as an essential complement¹⁰. Distributed under the Boost Software License, Clipper2 executes polygon boolean unions, intersections, and Minkowski dilation/erosion offsets¹⁰. Operating on 64-bit integer coordinate spaces, it provides absolute numerical robustness against degenerate vertex conditions, allowing fragmented cross-sectional line collections to be expanded, closed, and fused into continuous, unified wall boundaries¹⁰.

3. The Algorithmic Slicing and Vector Extraction Pipeline

The pipeline converting raw, unstructured 3D inputs into clean 2D architectural vectors

proceeds through five sequential algorithmic stages: spatial normalization, horizontal section cutting, planar segmentation, polygon regularization, and topological graph assembly.

Spatial Normalization and Floor Plane Alignment

Imported 3D assets frequently arrive with unpredictable world coordinates, irregular scaling, and arbitrary angular tilt. To establish a standardized coordinate frame, the pipeline computes the scene's primary bounding envelope and determines its upward-pointing normal vector. When processing photogrammetric point clouds, ground surface extraction is performed using three-point planar Random Sample Consensus (RANSAC)⁶. The dominant horizontal plane is

transformed via rigid affine rotation to coincide with the $z = 0$ floor datum, establishing the building up-vector as $n = (0, 0, 1)$. Vertical ceiling extents are determined by isolating the highest concentration of parallel planar points, establishing the total vertical elevation

$$H = z_{\max} - z_{\min}.$$

Cross-Sectional Plane Slicing

Architectural convention dictates that standard 2D floor plans represent an eye-level horizontal section cut below the ceiling and above low-profile clutter, typically positioned at:

$$z_{\text{cut}} = z_{\min} + 1.2 \text{ m to } 1.5 \text{ m}$$

Passing the cutting plane at this elevation intersects door apertures, window glazing, interior partitions, and perimeter facades while passing cleanly beneath overhead structural beams. For polygonal boundary representation (B-rep) meshes, every triangular face comprising

vertices (v_0, v_1, v_2) is evaluated against the horizontal slicing plane $P : z - z_{\text{cut}} = 0$. When a triangle straddles the plane, the intersection between each intersected triangle edge and the cutting datum is determined through linear parametric interpolation:

$$t = \frac{z_{\text{cut}} - z_a}{z_b - z_a}, \quad p = v_a + t(v_b - v_a)$$

This evaluation yields an unordered collection of coplanar line segments on the (x, y) coordinate plane. In the Manifold library, this operation is executed against a parallelized Bounding Volume Hierarchy (BVH), yielding sorted polygonal loops that preserve interior-versus-exterior winding topologies⁷.

Point Cloud Planar Extraction

When processing dense laser scans or unmeshed point clouds, surface edge intersections cannot be evaluated directly. The system instead establishes a narrow horizontal sampling slab

centered at $[z_{\text{cut}} - \delta, z_{\text{cut}} + \delta]$, where $\delta \approx 0.05 \text{ m}$, projecting all interior points

orthogonally onto the 2D plane⁶.

Linear structures are identified using iterative 2D RANSAC line fitting⁶. Candidate lines are modeled via the Hessian normal form:

$$x \cos \theta + y \sin \theta - \rho = 0$$

Points falling within a Euclidean distance threshold of the candidate line are registered as inliers. These inliers are extracted as structural segments and removed from the active sampling set, repeating the process until remaining unclustered points fall below a minimum density threshold⁶.

Regularization and Manhattan-World Snapping

Raw extracted segment collections exhibit geometric noise, micro-fractures, collinear fragmentation, and scanning drift. Because standard architecture follows orthogonal structural systems, the segments undergo regularization enforcing the Manhattan World Assumption, which dictates that structural walls meet at right angles, or the Atlanta World Assumption, which supports multiple non-orthogonal dominant axes²⁸.

Collinear segments within an ϵ -proximity envelope are merged, and intermediate redundant vertices are eliminated using the Douglas-Peucker simplification algorithm. The angular

orientation $\theta = \arctan 2(\Delta y, \Delta x)$ of each remaining segment is compared against dominant directional vectors. Segments deviating from canonical orthogonal orientations by

less than a tolerance threshold (typically 5°) are snapped to parallel and perpendicular axes. Because slicing hollow, double-sided surface meshes produces disjoint interior and exterior wall boundaries, Clipper2 executes a polygon buffering operation (ClipperOffset) with mitered joins¹⁰. This operation dilates the segments across the standard architectural wall thickness

interval (100 mm to 250 mm), performing a boolean union that transforms fractured outlines into solid, continuous structural profiles¹⁰.

4. Parametric Data Structures for Interactive Web CAD Manipulation

Static SVG paths or flattened vector polylines are insufficient for an interactive design platform. A user manipulating a floor plan expects parametric behavior: dragging a wall junction must stretch attached walls, preserve corner miter angles, maintain relative door and window positions, and update room dimensions in real time¹⁸.

The Planar Topological Graph Model

To support parametric reactivity, the application architecture encapsulates spatial elements within an explicit Planar Node-Edge Graph.

Vertices represent junction nodes $V_i = (x_i, y_i)$ where wall segments terminate or

intersect¹⁷. Edges represent parametric walls $E_{ij} = (V_i, V_j)$ bounding the spaces¹⁷. Edge attributes define the wall's metric thickness T , its overall height H , its internal construction layers (such as structural framing, thermal insulation, or drywall finish), and a collection of hosted sub-elements¹⁸. Sub-elements such as doors, windows, and passage openings are defined parametrically rather than via global Cartesian coordinates¹⁸. Each opening stores a normalized linear scalar $u \in [0, 1]$ relative to the host edge centerline:

$$P_{\text{opening}}(u) = V_i + u \cdot (V_j - V_i)$$

When a user translates a junction vertex V_i across the canvas, the hosting wall stretches and rotates. Because the sub-elements are anchored to the unit scalar u , doors and windows traverse with the wall's updated trajectory, avoiding manual repositioning and preventing geometry detachment¹⁸.

Dynamic Miter Calculation at Multi-Wall Junctions

Rendering walls as simple rectangular bounding volumes creates overlapping geometry or visible gaps at angled intersections¹⁸. A parametric CAD engine resolves these intersections by calculating corner miter line bisectors at every active graph vertex¹⁸.

For two wall centerlines intersecting at node V_k with normalized directional vectors d_1 and d_2 , the unit bisector vector b is computed as:

$$b = \frac{d_1 - d_2}{\|d_1 - d_2\|}$$

Given a nominal wall thickness w , the offset distance from the junction vertex to the corner vertex is governed by the half-angle $\phi = \frac{1}{2} \arccos(d_1 \cdot d_2)$:

$$L_{\text{miter}} = \frac{w/2}{\sin \phi}$$

When an interaction event moves vertex V_k , the engine traverses the incident edges in the topological graph, recalculates the corner miter bisectors, and re-renders the wall boundary

contours within the active animation frame¹⁸.

Automated Room Cycle Extraction

Displaying live room square footage and rendering floor material fills requires extracting enclosed spatial boundaries from the edge network. The system constructs a Doubly Connected Edge List (DCEL) or Half-Edge structure from the active graph¹⁵. Directed half-edges are sorted cyclically by polar angle at each vertex, enabling clockwise and counter-clockwise cycle traversal to isolate interior room faces from the exterior boundary¹⁷. The continuous metric surface area for each identified room polygon

$P = \{(x_0, y_0), \dots, (x_{n-1}, y_{n-1})\}$ is computed using the Shoelace formula:

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|$$

This area calculation updates dynamically during canvas drag interactions, ensuring that room schedules and dimensions remain synchronized with geometric edits¹⁹.

5. Architectural Trade-Offs: Client WebAssembly Versus Distributed Backend

Determining where computation should execute requires evaluating browser runtime constraints against server infrastructure costs and network bandwidth limits.

System Characteristic	Client-Side WebAssembly (Wasm)	Python Backend (FastAPI / Celery)	C++ / .NET Native Microservices
Compute Execution	Client-side CPU execution via SIMD-compiled Wasm modules ⁷ .	Multi-process worker pools (CPython GIL-bound; relies on C extensions).	Multithreaded native binaries with AVX-512 and OpenMP scaling ⁷ .
Bandwidth Overhead	Zero upload bandwidth. Files are processed locally via the browser File API ¹ .	High. Raw 3D files (50 MB–500 MB) must be uploaded before slicing can begin.	High. Assets must cross network boundaries to reach processing nodes.
Memory	Constrained by	Scalable virtual	High memory

Boundaries	browser sandbox limits (typically 2 GB–4 GB per Wasm heap).	memory allocations limited only by server node sizing.	efficiency; fine-grained control over raw pointer allocations.
Algorithmic Ecosystem	Strong geometry kernels (manifold-3d, Clipper2 Wasm) ⁷ .	Access to machine learning and vision libraries (Open3D, PyTorch, SciPy) ⁶ .	Direct integration with industrial CAD frameworks (Open CASCADE, CGAL, Manifold) ⁷ .
Operational Cost	Zero server compute cost; scales with user base without infrastructure expansion.	High continuous operating expenses for GPU/CPU cloud worker instances.	Moderate-to-high cloud hosting and instance scaling costs.

Relying entirely on a client-side or backend architecture introduces operational bottlenecks. An asset exceeding 20 million points can crash a browser WebAssembly heap, while streaming 200 MB files to a cloud backend introduces severe network latency and unsustainable bandwidth costs.

The optimal architecture implements a tiered hybrid execution model:

- **Tier 1 (Client WebAssembly for Meshes < 50 MB):** Standard triangle meshes (GLTF, OBJ, IFC) are parsed directly inside the browser using manifold-3d and Clipper2 compiled to WebAssembly⁷. Slicing executes locally in milliseconds, completely bypassing network transfer, eliminating cloud hosting expenses, and preserving complete user data privacy¹.
- **Tier 2 (Distributed Backend for Large Point Clouds and Deep Learning):** Massive point clouds (>50 MB PLY, LAS, E57) or workflows requiring complex deep learning inference are offloaded via streaming WebSocket or multipart upload to a containerized backend. A native C++ or Python service leveraging Open3D and PyTorch processes the data, returning a lightweight, structured JSON vector payload (often under 200 KB) back to the client canvas⁶.

6. Machine Learning Models for Structural Cleanup and Layout Prediction

While deterministic algorithmic slicing functions effectively on clean CAD geometry, real-world 3D scans frequently suffer from occlusions, non-watertight holes, and scanner noise. Specialized deep learning architectures bridge this gap by predicting clean architectural topology directly from unstructured spatial data.

Transformer-Based Layout Reconstruction: RoomFormer

RoomFormer approaches floor plan reconstruction as a direct set-prediction problem, bypassing brittle multi-stage heuristic pipelines³⁷. Given a top-down, birds-eye-view (BEV) 2D density and height projection map derived from an unorganized 3D point cloud or mesh, RoomFormer extracts multi-scale feature maps using a convolutional or transformer backbone³⁷.

The network's transformer decoder utilizes a hierarchical query schema: a higher-level query

set parameterizes distinct room polygons (M rooms), while nested lower-level queries

parameterize the sequence of ordered corner vertices (N corners per polygon)³⁷. Rather than outputting raster pixel segmentation masks that require noisy post-hoc contour tracing, RoomFormer directly predicts ordered 2D polygonal coordinates and architectural semantic classifications (e.g., bedroom, kitchen, wall, opening), which map seamlessly into parametric CAD node-edge graphs³⁷.

The model operates along a linear sequence of transformations:

- An unorganized 3D point cloud or mesh is projected into a top-down 2D birds-eye-view density map³⁷.
- A convolutional or transformer backbone extracts multi-scale spatial feature representations³⁷.
- A transformer encoder contextualizes global spatial relationships across the room footprint⁴⁰.
- A hierarchical decoder evaluates two-level queries to predict individual rooms and their associated corner vertices³⁷.
- The network outputs structured vector polygons containing semantic classifications for direct CAD graph ingestion³⁷.

Autoregressive Structured CAD Generation: SceneScript

SceneScript formulates architectural scene reconstruction through language modeling rather than direct numerical coordinate regression²⁹. The network processes point cloud tokens and autoregressively generates an explicit sequence of structured, human-readable CAD commands (such as `make_wall(x1, y1, x2, y2)` and `add_door(offset, width)`)²⁸.

Because output generation is governed by language grammar rules, the model enforces structural connectivity by construction²⁹. This design eliminates geometric artifacts common to unconstrained regression models, such as floating doors, disconnected wall segments, and non-manifold boundaries²⁹.

7. Programmatic Orchestration via the Cursor SDK

The Cursor SDK provides programmatic access to agentic workflows, operating via local runtime hooks or managed cloud virtual machines²². It is essential to recognize the architectural boundary of the Cursor SDK: it is not a high-performance computational geometry kernel or an embedded matrix solver, but an autonomous coding and scripting agent harness²². Within a floor plan platform, the Cursor SDK is deployed across two architectural

tiers: natural language CAD manipulation and automated geometric schema repair.

Tier A: Natural Language CAD Manipulation

The Cursor SDK can be embedded within the platform backend to act as an intelligent architectural copilot:

- The web frontend serializes the active floor plan graph into an architectural JSON schema and pairs it with a user prompt (such as a request to divide a master suite into two rooms with an interior partition and a centered door).
- Operating within an isolated run context, the Cursor agent (@cursor/sdk or Python cursor-sdk) analyzes the schema, invokes structural calculation tools, and emits a verified JSON mutation patch²².
- The frontend ingests the mutation patch, updating its internal topological graph and re-rendering the canvas in real time.

Tier B: Automated Geometric Schema Repair and Code Generation

When unorganized slicing algorithms output topological errors—such as unclosed room cycles, overlapping collinear wall segments, or malformed junction nodes—the Cursor SDK inspects the scene graph data structure:

- The agent executes deterministic validation tools in a headless sandboxed execution environment²².
- If non-manifold edge loops or orphaned door primitives are detected, the agent autonomously executes repair routines, ensuring that the exported data adheres strictly to IFC or DXF structural standards¹.

8. Interactive Web CAD Frontend Engineering and 2D/3D Synthesis

The client frontend must maintain a responsive interface that enables users to drag wall nodes, insert components, adjust dimensions, and toggle seamlessly between 2D drafting and real-time 3D visualization.

Canvas Graphics Engine Selection

For 2D drafting interfaces, Konva.js (via react-konva) provides a robust foundation¹⁹. It implements an object-oriented canvas scene graph supporting high-performance shape caching, hit detection, z-indexing, and declarative data binding¹⁹.

Alternatively, developers can adopt a unified Three.js context for both 2D drafting and 3D visualization, following the architecture of open-source tools like OpenPlan3D²¹. By positioning an OrthographicCamera directly along the positive Z-axis with top-down controls, the system creates a hardware-accelerated 2D drafting plane¹. Switching to 3D perspective viewing simply requires interpolating the viewport to a PerspectiveCamera with orbit controls, eliminating the complexity of synchronizing state across separate 2D and 3D rendering engines¹.

Interaction Constraints and Snapping Systems

To ensure an intuitive drafting experience, the canvas interaction engine must calculate four snapping behaviors during node translation:

- **Grid Quantization:** Restricts raw pointer coordinates to metric or imperial increments (10 mm, 50 mm, 100 mm)¹.
- **Node-to-Node Fusion:** When a dragged vertex enters a defined screen radius ($\rho \leq 12 \text{ px}$) of an existing vertex, it snaps to that location, fusing the vertices into a shared multi-wall junction¹⁹.
- **Collinear Alignment Rays:** Projects alignment rays along the trajectories of adjacent walls, snapping moving nodes to collinear paths and displaying dynamic visual guides.
- **Orthogonal Angle Locks:** Clamps wall angles to 0° , 45° , or 90° increments relative to the origin node during drafting operations³⁴.

Real-Time Extrusion (2D-to-3D Synthesis)

Whenever the user modifies the 2D planar graph, the corresponding 3D representation updates synchronously:

- The 2D boundary polygons of the walls (derived from Clipper2 miter calculations) are converted into THREE.Shape paths¹⁰.
- Wall openings (doors and windows) are treated as negative volumetric cutouts or subtracted via CSG boolean operations using manifold-3d⁷.
- THREE.ExtrudeGeometry extrudes the resulting shape vertically along the Z-axis according to the wall's parametric height parameter H ²¹.
- Floor slabs are generated directly by triangulating the room interior cycle polygons using standard Earcut algorithms, immediately rendering material textures (wood, tile, concrete) across the floor surfaces³⁴.

9. System Implementation Matrix and End-to-End Lifecycle

Deploying this architecture requires organizing the subsystem components into a coherent operational pipeline that balances computational efficiency, data integrity, and user experience.

Subsystem Component	Operational Environment	Core Technological Dependency	Primary Functional Responsibility	Ingestion / Output Format
Ingestion & Slicing	Client Browser / Worker (Tier 1)	manifold-3d (Wasm) ⁷	Fast horizontal cross-section cutting at $z = 1.2 \text{ m}$ ¹⁵ .	Ingests GLTF/OBJ/IFC; emits raw 2D cross-section lines ⁷ .

Scan Preprocessing	Backend Cloud Worker (Tier 2)	Open3D / RANSAC C++ ⁶	Ground plane leveling, spatial filtering, and line extraction ⁶ .	Ingests dense PLY/LAS scans; emits 2D segment clusters ⁶ .
Geometric Cleanup	Client / Backend Shared	Clipper2 / Pyclipr ¹⁰	Polygon offsetting, air-gap fusion, and miter joins ¹⁰ .	Ingests raw line segments; emits closed wall polygon boundaries ¹⁰ .
Parametric Model	Client Application State	Custom TypeScript Graph ¹⁷	Managing junction nodes, parametric edges, and room cycles ¹⁷ .	Ingests polygons; maintains live reactive JSON scene graph.
2D/3D Viewports	Web Client UI	Konva.js & Three.js ¹⁹	Interactive CAD manipulation, snapping, and 3D extrusion ¹⁹ .	Ingests scene graph; renders canvas and WebGL/WebGPU scenes.
AI Copilot & Repair	Isolated Backend Runner	Cursor SDK (@cursor/sdk) ²⁴	Schema validation, non-manifold repair, natural language edits ²² .	Ingests graph JSON + text prompts; emits validated JSON patches.
Document Exporter	Client / Server Shared	IfcOpenShell / dxf-writer [cite: 9]	Generating standardized architectural drawing deliverables ¹ .	Ingests scene graph; exports DXF, IFC, SVG, and PDF assets ¹ .

The complete operational lifecycle begins when a user drops an asset into the browser interface. The platform inspects the file: if it is a standard polygonal mesh under 50 MB, the browser executes the local WebAssembly pipeline via manifold-3d, computing a horizontal cross-section at $z = 1.2 \text{ m}$ in milliseconds⁷. If the asset is a dense point cloud exceeding 50 MB, it streams asynchronously to the backend worker service for RANSAC planar extraction

and line clustering⁶.

Once the raw intersecting segments are extracted, Clipper2 buffers and unites the geometry, removing co-planar artifacts and closing disjoint boundaries¹⁰. The resulting contours are mapped into a Planar Node-Edge Graph, establishing explicit wall junctions, calculating corner miter bisectors, and detecting closed room boundaries¹⁷.

The user interacts with the generated layout in the browser workspace: dragging wall nodes, snapping corners to perpendicular axes, inserting doors and windows with dynamic offsets, and assigning room labels¹⁹. Natural language commands can be dispatched via the embedded Cursor SDK interface, directing an agent to perform high-level architectural modifications, repair non-manifold rooms, or run layout optimization checks²². Finally, the validated parametric model exports directly to CAD formats (DXF), Building Information Modeling containers (IFC), scalable vectors (SVG), or print-ready architectural PDF drawings¹.

Works cited

1. OpenPlan3D: Free Floor Plan Editor — 2D & 3D, Open Source, <https://openplan3d.com/>
2. OpenPlan3D Capture — Scan Floor Plans with iPhone LiDAR, <https://openplan3d.com/capture>
3. WWDC23 — Explore enhancements to RoomPlan - Medium, <https://medium.com/@ganeshrajugalla/wwdc23-explore-enhancements-to-room-plan-7ecbb8f68328>
4. RoomPlan | Apple Developer Documentation, <https://developer.apple.com/documentation/roomplan>
5. AR room planning: How it works, tools & accuracy - Netguru, <https://www.netguru.com/blog/augmented-reality-room-planning-technologies>
6. GitHub - Parikshit00/Deep3D-FloorPlan-Net: Deep 3D Semantic, <https://github.com/Parikshit00/Deep-3D-Semantic-Segmentation-and-Floor-Plan-Extraction-RandLANet->
7. elalish/manifold: Geometry library for topological robustness - GitHub, <https://github.com/elalish/manifold>
8. Create a 3D Floor Plan of an Interior Room Using RoomPlan, <https://technostacks.com/blog/create-a-3d-floor-plan-of-an-interior-room-using-roomplan/>
9. GitHub - joyjo/to-generate-2D-floorplan-CAD-from-3D-point-clouds, <https://github.com/joyjo/to-generate-2D-floorplan-CAD-from-3D-point-clouds>
10. Clipper2 - Polygon Clipping Offsetting & Triangulating - angusj.com, <https://www.angusj.com/clipper2/Docs/Overview.htm>
11. A Semantic Scan-to-IFC Pipeline for Automated Generation of BEM, <https://www.mdpi.com/2075-5309/16/15/3036>
12. Overview - angusj.com, http://www.angusj.com/delphi/clipper/documentation/Docs/Overview/_Body.htm
13. Python Libraries for Mesh, Point Cloud, and Data Visualization (Part 1), <https://towardsdatascience.com/python-libraries-for-mesh-and-point-cloud-visu>

- [alization-part-1-daa2af36de30/](#)
14. manifold3d - PyPI, <https://pypi.org/project/manifold3d/>
 15. cartesian-theatrics/clj-manifold3d: Manifold (3D) wrapper for Clojure, <https://github.com/cartesian-theatrics/clj-manifold3d>
 16. pyclipr · PyPI, <https://pypi.org/project/pyclipr/>
 17. Computer-Aided Architectural Design. "Hello, Culture": 18th, <https://dokumen.pub/computer-aided-architectural-design-quotello-culturequot-18th-international-conference-caad-futures-2019-daejeon-republic-of-korea-june-2628-2019-selected-papers-1st-ed-978-981-13-8409-7978-981-13-8410-3.html>
 18. SketchUp Wall Modeling: The Complete Guide for Architects in 2026, <https://muzwalls.com/blog/sketchup-wall-modeling-complete-guide-architects-2026>
 19. ahmethakanbesel/floor-maker - GitHub, <https://github.com/ahmethakanbesel/floor-maker>
 20. A construction-style web app (like building a cabin) - three.js forum, <https://discourse.threejs.org/t/a-construction-style-web-app-like-building-a-cabin/87094>
 21. About OpenPlan3D | Free Open Source Home Design Tool, <https://openplan3d.com/about>
 22. Cursor SDK Complete Guide: Coding Agents Evolved into, https://note.com/snake_dragon/n/nc9ae86cff8b4?hl=en
 23. Cursor Python SDK | Cursor Documentation, <https://cursor.com/docs/sdk/python>
 24. Cursor SDK Tutorial: Run Coding Agents With TypeScript - DataCamp, <https://www.datacamp.com/tutorial/cursor-sdk>
 25. Manifold Library · elalish/manifold Wiki - GitHub, <https://github.com/elalish/manifold/wiki/Manifold-Library/ee3eee76533de1223af38a45e1bc430223c71af1>
 26. Solving a 3D Modelling Nightmare - Detecting & Fixing Input Geometry, <https://www.rocscience.com/learning/solving-a-3d-modelling-nightmare-detecting-fixing-input-geometry>
 27. Blog - lukeparry.uk, <https://lukeparry.uk/category/blog/>
 28. PolyLayout: Multi-room Manhattan Layout Estimation - OpenReview, <https://openreview.net/pdf/7a065b2f1fc1ad88a2355d912352923e0f18215b.pdf>
 29. SceneScript: Reconstructing Scenes With An Autoregressive ... - arXiv, <https://arxiv.org/html/2403.13064v1>
 30. BIM handbook: A guide to building information modeling for owners, https://www.academia.edu/3183272/BIM_handbook_A_guide_to_building_information_modeling_for_owners_managers_designers_engineers_and_contractors
 31. An algorithm for inflating/deflating (offsetting, buffering) polygons, <https://stackoverflow.com/questions/1109536/an-algorithm-for-inflating-deflating-offsetting-buffering-polygons>
 32. Construction of 3D Volumetric Objects for a 3D Cadastral System, https://gdmc.nl/publications/2015/Construction_3D_Volumetric_Objects.pdf
 33. Manifold Library · elalish/manifold Wiki - GitHub,

- <https://github.com/elalish/manifold/wiki/Manifold-Library>
34. Features | Free 2D/3D Floor Plan Software - OpenPlan3D,
<https://openplan3d.com/features>
 35. mesh-simplification · GitHub Topics,
<https://github.com/topics/mesh-simplification?l=python&o=desc&s=stars>
 36. Javascript Clipper download | SourceForge.net,
<https://sourceforge.net/projects/jsclipper/>
 37. [CVPR 2023] RoomFormer: Two-level Queries for Single-stage,
<https://github.com/ywyue/roomformer>
 38. OpenShape3D vs. Shapr3D: An Honest Comparison,
<https://openshape3d.com/articles/openshape3d-vs-shapr3d>
 39. Yuanwen Yue ywyue - GitHub, <https://github.com/ywyue>
 40. Floorplan Reconstruction Using Two-Level Queries - Yuanwen Yue,
<https://ywyue.github.io/RoomFormer/>
 41. arXiv:2407.10439v1 [cs.CV] 15 Jul 2024, <https://arxiv.org/pdf/2407.10439>
 42. SpatialLM: Training Large Language Models for Structured Indoor,
<https://arxiv.org/html/2506.07491v1>
 43. GitHub - cursor/cookbook, <https://github.com/cursor/cookbook>
 44. Free Drawing and Painting on HTML5 Canvas with JavaScript | Konva,
https://konvajs.org/docs/sandbox/Free_Drawing.html
 45. Open Plan 3D (thelodgebots/open3dfloorplan) | Context7,
<https://context7.com/thelodgebots/open3dfloorplan>